Grandma's Building Blocks

Time Limit: 1 seconds
Memory Limit: 32 MB

Problem Description

In Mia's attic, there is an old box of wooden blocks left behind by her grandmother. Each block is carved with a distinct number, and grandma used to stack them into a tower shaped like a binary tree. Every time she built a tower, she wrote down the order in which the blocks were placed — but only two of the three classic orders, never all three.

One day, while cleaning the attic, Mia finds an old notebook. On one page, grandma wrote down the **preorder** and **postorder** sequences of a tower she once built:

Preorder: 1 2 3
Postorder: 3 2 1

Mia tries to redraw the tower from these two sequences, but soon realizes something strange: there is more than one tower shape that matches **both** sequences!

Mia decides she still wants a definite tower to look at, even when the notes are ambiguous. So she comes up with a simple rule: whenever a node's children cannot be fully determined from the sequences (i.e., the node turns out to have only one child), she will always attach that single child as the **left** child. This way, she can always draw exactly one tower from any valid pair of notes, and she can separately keep track of whether that tower is the **only** possible one.

Now, please help Mia write a program that, given the preorder and postorder sequences of a binary tree, reconstructs a tower using the rule above, and reports:

1. the **inorder** traversal of the reconstructed tower, and
2. whether the reconstructed tower is the **unique** tree consistent with the given preorder and postorder sequences.

A Warm-up Example

Consider the example above: preorder = $[1, 2, 3]$, postorder = $[3, 2, 1]$.

Here are **two different** binary trees, both of which have preorder $[1, 2, 3]$ and postorder $[3, 2, 1]$:

Tree A: 1 → left child 2 → left child 3 Tree B: 1 → right child 2 → right child 3

Tree A's inorder traversal is 3 2 1, while Tree B's inorder traversal is 1 2 3 — two different trees, two different inorder results, from the exact same preorder and postorder sequences. That is what it means for this pair of sequences to be **not unique**.

Since more than one tree is possible, Mia's rule kicks in: whenever a node's child is ambiguous like this, always attach it as the **left** child. This selects Tree A, so Mia's program should output inorder 3 2 1 together with `NOT UNIQUE`.

Input

The first line contains a single integer n, the number of blocks (nodes) in the tower.

The second line contains n space-separated integers, the **preorder** traversal of the tower.

The third line contains n space-separated integers, the **postorder** traversal of the tower.

Output

Output two lines:

- The first line contains n space-separated integers — the **inorder** traversal of the tower reconstructed using Mia's rule (single child ⇒ left child).
- The second line contains `UNIQUE` if the reconstructed tower is the only binary tree consistent with the given preorder and postorder sequences, or `NOT UNIQUE` otherwise.

Constraints

- $1 \leq n \leq 3000$
- All n values are distinct **positive** integers ($1 \leq \text{value} \leq 10^9$).
- It is guaranteed that the given preorder and postorder sequences correspond to **at least one** valid binary tree.

Hint

- Remember Mia's rule: whenever a node's child cannot be fully determined from the two sequences, attach it as the **left** child.

- Since $n \leq 3000$, an $O(n^2)$ approach (e.g. scanning a segment to locate a value) is fast enough. You do **not** need any hashing-based lookup to pass this problem.

Sample Testcase 1

Sample Input 1

```
3
1 2 3
3 2 1
```

Sample Output 1

```
3 2 1
NOT UNIQUE
```

Explanation of Sample 1

This is exactly the warm-up example above: both Tree A ($1 \to 2 \to 3$, all left children) and Tree B ($1 \to 2 \to 3$, all right children) have preorder 1 2 3 and postorder 3 2 1, but their inorder traversals differ (3 2 1 versus 1 2 3). Since two different trees fit the same input, the answer is `NOT UNIQUE`, and Mia's left-child rule selects Tree A, giving inorder 3 2 1.

Sample Testcase 2

Sample Input 2

```
5
1 2 4 5 3
4 5 2 3 1
```

Sample Output 2

```
4 2 5 1 3
UNIQUE
```

Sample Testcase 3

Sample Input 3

```
1
7
7
```

Sample Output 3

```
7
UNIQUE
```